# Topic 15: OOP Design Principles and Practice Programs

## 15.1 SOLID Principles (Overview)

- S -- Single Responsibility: each class should have one reason to change
- O -- Open/Closed: open for extension, closed for modification (use virtual functions)
- L -- Liskov Substitution: derived objects must be substitutable for base objects without breaking the program
- I -- Interface Segregation: prefer many small interfaces over one large one
- D -- Dependency Inversion: depend on abstractions, not concrete classes

## 15.2 Common Design Patterns (Brief)

- Singleton: ensures only one instance exists -- private constructor, static getInstance()
- Factory Method: creates objects without specifying the exact class -- returns base class pointer
- Observer: one object (subject) notifies many observers automatically when state changes
- Strategy: encapsulates interchangeable algorithms behind a common interface

```
// Singleton example
class Logger {
    Logger() {}
public:
    static Logger& getInstance() {
        static Logger instance;
        return instance;
    }
    void log(const string& msg) { cout << "[LOG] " << msg << endl; }
};

// Usage: Logger::getInstance().log("App started");
```

## 15.3 Practice Program -- Library Management System

- Classes: Book, Member, Library
- Book: title, author, ISBN, isAvailable
- Member: name, memberId, list of borrowed books
- Library: collection of books and members, borrowBook(), returnBook(), searchBook()
- Apply encapsulation, inheritance (PremiumMember : Member), polymorphism (virtual canBorrow())

```
class Book {
    string title, author, isbn;
    bool   isAvailable = true;
public:
    Book(string t, string a, string i) : title(t), author(a), isbn(i) {}
    bool available()          { return isAvailable; }
    void checkout()           { isAvailable = false; }
    void checkin()            { isAvailable = true;  }
    string getTitle()         { return title; }
};
```

### 15.4 Practice Program -- Shape Hierarchy

- Abstract base: Shape with pure virtual area() and perimeter()
- Derived: Circle, Rectangle, Triangle, Square
- Store all shapes in a vector<Shape*> and compute total area
- Add a printAll() free function that works for any future shape without modification

```
vector<Shape*> shapes;
shapes.push_back(new Circle(5));
shapes.push_back(new Rectangle(4, 6));
shapes.push_back(new Triangle(3, 4, 5, 6));

double total = 0;
for (Shape* s : shapes) { s->draw(); total += s->area(); }
cout << "Total area: " << total << endl;

for (Shape* s : shapes) delete s;
```

---